

# Customer Feedback Sentiment Analysis using BERT

Luis Lopez-Echeto Bordon  
UCSC Extension Course: AISV.X401.(21)  
Summer 2025

# Problem Statement

## The Challenge

Businesses struggle to efficiently process vast amounts of customer feedback from surveys and reviews.

## The Goal

Develop an automated system to quickly classify customer feedback as positive, or negative.

## Impact

Help companies understand customer sentiment faster to improve products and services.

# Problem Statement: Why BERT?

---

**Contextual Understanding:** BERT grasps the full meaning of a sentence, avoiding the pitfalls of older models.

**Highly Accurate:** Its deep learning architecture provides reliable business insights.

**Efficient:** By using **transfer learning**, we can fine-tune a pre-trained model with a small amount of data.

# Project: Data Preprocessing

**Dataset:** Customer feedback from Kaggle.

**Challenge:** Data used a non-standard format with quotes wrapping each row and escaped quotes within the text.

```
csv
"Text, Sentiment, Source, Date/Time, User ID, Location, Confidence Score"
""I love this product!"" , Positive, Twitter, 2023-06-15 09:23:14, @user123, New York, 0.85
```

**Solution:** Built a custom parser to clean the text.

**Outcome:** The data structured as a pandas DataFrame, ready for analysis and modeling.

# Preparing Data for the BERT Model

- **Label Encoding:** Convert text sentiment labels into a numerical format.

- **Positive** reviews were mapped to 1.
- **Negative** reviews were mapped to 0.

| Text                      | Sentiment | label |
|---------------------------|-----------|-------|
| I love this product!      | Positive  | 1     |
| The service was terrible. | Negative  | 0     |
| This movie is amazing!    | Positive  | 1     |

- **Minimal Text Preprocessing:** We only stripped extra whitespace.
- **Why so little preprocessing?**
  - BERT handles tasks like lowercasing, punctuation removal, and stemming with its own internal tokenizer.
  - This simplifies our pipeline and preserves valuable context for the model.

# Preparing Data for Training

---

- **Split the Data:** We divided the dataset into an 80% training set and a 20% validation set. The training set teaches the model; the validation set tests it on new data.
- **Tokenization:** The `BertTokenizer` converts raw text into a numerical format. It also handles padding (making sentences the same length) and truncation (cutting long sentences).
- **Prepare for Pytorch:** A custom `torch.utils.data.Dataset` class was created to correctly format the tokenized data for training in batches.

# Model Training and Evaluation

- **The Process:** We fine-tune a pre-trained BERT model for our task. We define key parameters (epochs, batch size, etc.) using `TrainingArguments`.
- **The `Trainer`:** This class from Hugging Face automates the entire training loop. It efficiently handles all the complex steps, including data batching, calculating the loss, backpropagation, and updating the model's weights.
- **Evaluation:** After training, the model is evaluated on a separate validation set. This ensures we have an unbiased measure of its performance on new, unseen data.
- **Result:** The final result is a model ready to make fast and accurate sentiment predictions.

# Examples in Action: Positive Feedback

---

"I'm honestly impressed – I thought this would take days."

**Model Prediction:** Negative (0.56 confidence)

**Why?** The model misread the slightly sarcastic or cynical tone, which highlights the need for a human-in-the-loop system to validate nuanced language.

"It's working now."

**Model Prediction:** Positive (0.85 confidence)

**Why?** The model assigned high confidence to a brief, positive-sounding statement, even though the user was likely just stating a fact. This reminds us that context is key for short, simple replies.



# Examples in Action: Negative Feedback

---

## Negative Feedback:

"Please escalate this – I've emailed three times and still no reply."

**Model Prediction:** Negative (0.73 confidence)

**Why?** The model correctly flagged frustration and urgency based on keywords like "escalate" and phrases indicating repeated attempts without a solution. This is ideal for an automated triage system.

"Still no resolution. Starting to regret choosing your platform."

**Model Prediction:** Negative (0.86 confidence)

**Why?** The model correctly identified a high degree of negative sentiment, indicating a user who is likely to churn. This is critical for customer retention.

# Project Summary

---

- **The Solution:** We built a robust sentiment analysis system by fine-tuning a pre-trained **BERT model**.
- **The Outcome:** The model transforms unstructured customer feedback into valuable, actionable business insights with high accuracy.
- **The Limitation:** Our model, while powerful, can still struggle with complex human nuances like sarcasm.
- **The Best Approach:** The ideal solution is a "**human-in-the-loop**" system, where the AI flags ambiguous feedback for a human to review, ensuring complete accuracy.

---

Thanks! 🎉

Github: [elchicha](#)